

Distributed MPC and Secure Aggregation Testbed on Raspberry Pi

1. Abstract

This report documents the design, implementation, debugging, and experimental evaluation of a Raspberry Pi based smart-meter privacy-preserving aggregation testbed. The system began as a C-based multi-party computation benchmark using additive secret sharing and TCP communication, then evolved into a Linux-hosted smart-meter emulator and finally into a high-N secure aggregation stress test with 1000 simulated smart meters.

The testbed uses two Raspberry Pis connected over Tailscale. Pi 5 acts as the higher-capacity host and Pi 2 acts as a lower-capacity peer. PC 5 is used as the development and orchestration machine. The work validates several increasingly demanding configurations: 2-party distributed MPC, 6-party distributed MPC, CSV-backed smart-meter emulation, constrained CPU and memory execution on Pi 5, automated benchmark collection, and a 1000-meter central secure aggregation stress test with proportional load placement.

The core experimental conclusion is that the C-based MPC implementation correctly reconstructs aggregates across real distributed Raspberry Pis for the tested 2-party and 6-party configurations. The smart-meter emulator remains correct under CPU and memory constraints. The high-N secure aggregation path successfully processed 1000 simulated constrained meters in two full runs, receiving all submissions and reconstructing the expected aggregate. A later 1000-meter run failed due to launch or timeout behavior, showing that the high-concurrency path is promising but not yet fully reliable without further orchestration hardening and repeated trials.

This report distinguishes between the all-party MPC benchmark and the later central secure aggregation stress path. The former validates share exchange and all-party reconstruction. The latter evaluates high-N masked submission to a central aggregator and is closer to a smart-meter fleet aggregation model. The current work is a Linux-hosted distributed validation platform, not an embedded STM32/MSP430 deployment.

2. System Objective

The project objective is to validate a privacy-preserving smart-meter aggregation system on constrained distributed hardware. A smart-grid scenario contains many meters, each with a private reading. The desired computation is an aggregate such as the total energy consumption across meters, without revealing each individual reading directly.

The work pursued this objective in stages:

1. Establish reliable network communication between Raspberry Pis.
2. Validate the existing C MPC primitives across physical devices.

3. Convert benchmark-style secret inputs into smart-meter-style CSV inputs.
4. Refactor protocol execution into reusable code.
5. Measure correctness and latency under resource constraints.
6. Automate benchmark runs and result collection.
7. Explore high-N viability using a central secure aggregation stress test.

The underlying research question is:

Can a low-cost two-Pi testbed execute privacy-preserving smart-meter aggregation correctly under constrained resources, and can it emulate large numbers of smart meters well enough to evaluate system viability?

The current answer is bounded but positive. The testbed is correct and operational for small all-party MPC experiments and can run a 1000-meter secure aggregation stress path in successful trials. It still requires larger-scale repeated testing, network impairment testing, dropout handling, and embedded transport work.

3. Hardware and Machine Roles

The testbed used three main machines.

Machine	Role	Address
PC 5	Pop!_OS development and orchestration machine	Local development host
Pi 5	Raspberry Pi 5, high-capacity Raspberry Pi node	Tailscale IP <code>100.101.20.65</code>
Pi 2	Raspberry Pi 2, lower-capacity Raspberry Pi node	Tailscale IP <code>100.85.83.7</code>

The repository path conventions were:

Machine	Repository path
PC 5	<code>~/Desktop/Work/Hanze/work/mpc-smartgrid</code>
Pi 5	<code>~/mpc-smartgrid</code>
Pi 2	<code>~/mpc-smartgrid</code>

A critical deployment rule was established after a binary mismatch bug:

PC 5 edits and syncs source code only.
 PC 5 must not sync build/ to the Pis.
 Each Pi must compile its own ARM binary locally.

This was necessary because PC 5 is x86_64 while the Pis are ARM devices. Copying PC-built binaries to the Pis caused execution failures.

4. Network Setup and Debugging

The initial practical blocker was Raspberry Pi communication. Tailscale was used to connect devices across networks. Early behavior was asymmetric: one device could reach another, but reverse connections would timeout. This caused SSH, ping, and netcat tests to behave inconsistently.

Several symptoms were observed:

```
Pi 2 -> Pi 5 worked through Tailscale.  
Pi 5 -> Pi 2 failed or timed out.  
PC 5 -> Pi 2 also failed initially.
```

Debugging steps included:

1. Checking `tailscale status`.
2. Checking `tailscale ping`.
3. Identifying expired or stale node keys.
4. Re-authenticating devices.
5. Ensuring both Pis were in the intended tailnet.
6. Using a temporary reverse SSH tunnel when Pi 2 could initiate outbound connections but could not accept inbound ones.
7. Regenerating/forcing Pi 2 into the correct tailnet identity.
8. Confirming final bidirectional Tailscale reachability.

The final working IP map was:

```
Pi 5 = 100.101.20.65  
Pi 2 = 100.85.83.7
```

After repair, both directions worked:

```
Pi 5 -> Pi 2: tailscale ping succeeded.  
Pi 2 -> Pi 5: tailscale ping succeeded.
```

This network recovery was a prerequisite for every later distributed experiment.

5. Codebase Structure

The final codebase includes the original MPC library, benchmark drivers, smart-meter emulation, secure aggregation stress binaries, and orchestration scripts.

File or directory	Purpose
<code>src/mpc_field.c</code>	Finite-field arithmetic
<code>src/mpc_share.c</code>	Additive secret sharing
<code>src/mpc_comm_tcp.c</code>	TCP communication backend
<code>include/mpc_runner.h</code>	Shared MPC runner interface
<code>src/mpc_runner.c</code>	Reusable MPC protocol execution logic
<code>test/bench_mpc.c</code>	Benchmark CLI for synthetic secret inputs
<code>apps/meter_node.c</code>	CSV-backed smart-meter emulator
<code>apps/secure_agg_server.c</code>	Central secure aggregation collector
<code>apps/secure_agg_meter.c</code>	Simulated secure aggregation meter client
<code>configs/cluster_test2.json</code>	2-party Pi 5/Pi 2 cluster config
<code>configs/cluster.json</code>	6-party distributed cluster config
<code>scripts/bench_worker.sh</code>	Per-Pi benchmark worker
<code>scripts/run_suite_pc5.sh</code>	PC 5 automated constraint benchmark suite
<code>scripts/run_scale_suite_pc5.sh</code>	N-scaling benchmark script
<code>scripts/secure_agg_launch_range.sh</code>	Range launcher for secure aggregation meters
<code>scripts/ run_final_1000_secure_agg_pc5.sh</code>	Final proportional 1000-meter secure aggregation orchestrator

The working build targets are:

```
build/test_field
build/test_share
build/test_uc1
build/bench_mpc
build/meter_node
build/secure_agg_server
build/secure_agg_meter
```

6. MPC Theory Used in the Testbed

6.1 Additive Secret Sharing

The original MPC testbed uses additive sharing over a finite field. A secret value `x` is split into `N` shares:

$$x = s_0 + s_1 + \dots + s_{\{N-1\}} \bmod p$$

Each party receives one share. Individually, a share does not reveal the original value. When all shares are summed modulo the field prime, the original value is reconstructed.

For aggregation, each party has its own private value. Party i splits its value into N additive shares and distributes share j to party j . Each party receives one share from every other party and sums them into a local partial value. The sum of all local partial values equals the global aggregate.

6.2 Communication Pattern in the All-Party MPC Benchmark

The all-party benchmark uses pairwise communication between every party pair.

For N parties:

$$\text{Number of party pairs} = N(N - 1) / 2$$

The benchmark performs two exchange phases:

1. Share exchange.
2. Partial-sum exchange for reconstruction.

The communication cost grows approximately quadratically in the number of parties. This matters because the bottleneck for high- N all-party MPC is likely to be network and process orchestration, not local field arithmetic.

This is why 2-party and 6-party correctness results do not automatically imply 1000-party all-party MPC viability.

6.3 Smart-Meter Emulator Model

The `meter_node` binary uses the same MPC runner but replaces artificial CLI secrets with CSV-backed meter readings. For example:

```
timestamp,meter_id,energy_wh
2026-05-31T18:30:00Z,0,312
```

This makes the process behave like a smart-meter emulator:

```
read local meter value
convert value to finite-field element
```

```
execute MPC protocol
log aggregate, latency, and memory usage
```

The smart-meter emulator is still a Linux process. It is not yet a microcontroller implementation.

6.4 Central Secure Aggregation Model

The 1000-meter test does not use the same all-party reconstruction protocol. It uses a central secure aggregation model.

Each meter computes a masked value:

```
masked_i = reading_i + masks_added_by_i - masks_subtracted_by_i mod p
```

Every pair of meters has a deterministic pairwise mask. One meter adds the mask and the other subtracts it. When all masked values are summed by the aggregator, the masks cancel:

```
sum(masked_i) = sum(reading_i) mod p
```

The aggregator receives only masked submissions and reconstructs the global sum. Individual clear readings are not sent to the collector.

The current implementation uses deterministic mask derivation from a shared mask key supplied by the test harness. This is sufficient for stress testing the aggregation path, but it is not a production-grade key agreement protocol. A real deployment would need pairwise key establishment, dropout handling, replay protection, authentication, and collusion analysis.

7. Implementation Timeline

7.1 Communication Setup

The first major milestone was making Pi 5, Pi 2, and PC 5 communicate reliably over Tailscale. After several failed routes, stale identities, and SSH issues, Pi 2 was forced into a clean Tailscale state and assigned the usable IP:

```
Pi 2 = 100.85.83.7
```

Pi 5 remained:

```
Pi 5 = 100.101.20.65
```

Netcat and SSH tests confirmed TCP communication.

7.2 Existing Benchmark Validation

The existing C codebase built successfully on both Pis:

```
make clean  
make  
make test
```

Standalone field and sharing tests passed.

7.3 2-Party Distributed MPC

The first distributed MPC run used one party on each Pi:

```
Pi 5: secret = 300  
Pi 2: secret = 700  
Expected aggregate = 1000  
Observed aggregate = 1000
```

This proved that the real C MPC implementation could exchange shares between physical Raspberry Pis.

7.4 6-Party Distributed MPC

The next distributed test placed three party processes on each Pi:

```
Pi 5: nodes 0, 1, 2  
Pi 2: nodes 3, 4, 5
```

Inputs:

```
10, 20, 30, 40, 50, 60
```

Expected aggregate:

```
210
```

The run succeeded, proving that the testbed could host multiple independent MPC parties across the two physical machines.

7.5 Smart-Meter Emulator

The `meter_node` binary was added. It reads CSV input instead of accepting `--secret`.

Initial smart-meter test:

```
Pi 5 input = 312 Wh
Pi 2 input = 500 Wh
Expected aggregate = 812
Observed aggregate = 812
```

Repeated rounds also passed:

Round	Pi 5 input	Pi 2 input	Expected	Observed
0	312	500	812	812
1	318	520	838	838
2	305	510	815	815

7.6 MPC Runner Refactor

Originally, the benchmark logic lived inside `test/bench_mpc.c`. This was refactored into:

```
include/mpc_runner.h
src/mpc_runner.c
```

Both `bench_mpc` and `meter_node` now call the same reusable protocol execution path. This prevents the benchmark and smart-meter emulator from diverging.

7.7 Automated Constraint Benchmark Suite

An automated benchmark suite was added:

```
scripts/bench_worker.sh
scripts/run_suite_pc5.sh
```

It runs multiple 2-party smart-meter tests under different Pi 5 CPU and memory constraints, collects logs, and writes a CSV result matrix.

7.8 Final 1000-Meter Secure Aggregation Stress Path

Two new binaries were implemented:


```
secure_agg_server
secure_agg_meter
```

The server runs on Pi 5. Meter clients are distributed proportionally across Pi 5 and Pi 2. The final successful configuration used:

```
N = 1000 simulated meters
Pi 5 meters = 900
Pi 2 meters = 100
Per-meter memory cap = 16384 KB
Per-meter file descriptor cap = 64
Per-meter CPU time cap = 10 seconds
Pi 5 group cap = CPUQuota=300%, MemoryMax=1024M
Pi 2 group cap = CPUQuota=100%, MemoryMax=256M
```

8. Experimental Results

8.1 Core Correctness Results

Experiment	Expected aggregate	Observed aggregate	Status
2-party <code>bench_mpc</code>	1000	1000	PASS
6-party <code>bench_mpc</code>	210	210	PASS
2-party <code>meter_node</code>	812	812	PASS
Repeated smart-meter round 0	812	812	PASS
Repeated smart-meter round 1	838	838	PASS
Repeated smart-meter round 2	815	815	PASS

These results validate the basic correctness of the all-party MPC path for the tested sizes.

8.2 Manual Constrained Smart-Meter Run

A representative constrained run used:

```
Pi 5: CPUQuota=25%, MemoryMax=32M
Pi 2: normal peer
```

Device	Input	Observed aggregate	Protocol time	Max RSS
Pi 5	312	812	216.380 ms	6816 KB

Device	Input	Observed aggregate	Protocol time	Max RSS
Pi 2	500	812	300.018 ms	1700 KB

The expected aggregate was:

$$312 + 500 = 812$$

Both parties reconstructed the correct value.

8.3 Automated Constraint Matrix

The automated suite ran 12 2-party smart-meter cases. All passed.

Test ID	Pi 5 mode	Pi 5 value	Pi 2 value	Expected	Observed Pi 5	Observed Pi 2	Pi 5 ms	Pi 2 ms	Pi 5 RSS KB	Pi 2 RSS KB
T01	baseline	312	500	812	812	812	199.093	290.605	2624	2848
T02	baseline	318	520	838	838	838	230.059	276.792	2624	2848
T03	baseline	305	510	815	815	815	185.905	214.483	2624	2848
T04	CPUQuota=75%	312	500	812	812	812	201.044	260.798	6768	2764
T05	CPUQuota=50%	312	500	812	812	812	179.983	250.243	6768	2848
T06	CPUQuota=25%	312	500	812	812	812	186.379	259.674	6768	2848
T07	MemoryMax=64M	312	500	812	812	812	188.418	264.912	6816	2848
T08	MemoryMax=32M	312	500	812	812	812	771.406	842.532	6768	2848
T09	MemoryMax=16M	312	500	812	812	812	190.399	269.565	6768	2848
T10	CPUQuota=50%, MemoryMax=32M	312	500	812	812	812	182.713	270.667	6768	2720
T11	CPUQuota=25%, MemoryMax=32M	312	500	812	812	812	223.662	278.939	6768	2728
T12	CPUQuota=25%, MemoryMax=16M	312	500	812	812	812	196.825	224.780	6768	2848

8.4 Interpretation of Constraint Matrix

Correctness was preserved in every configuration.

CPU throttling down to 25% did not significantly degrade the 2-party workload. This suggests the tested workload is not CPU-bound. The dominant cost is likely communication and process orchestration rather than finite-field arithmetic.

Memory limits down to 16 MB did not break execution. Pi 5 constrained runs stayed near 6.7 MB RSS. Pi 2 stayed near 2.7–2.8 MB RSS.

The T08 MemoryMax=32M run showed a latency spike:

```
Pi 5 = 771.406 ms
Pi 2 = 842.532 ms
```

Since MemoryMax=16M ran faster, this should not be interpreted as a causal memory-limit effect. It is more likely transient scheduling or network jitter. More repeated trials are needed before drawing statistical latency conclusions.

8.5 Final 1000-Meter Secure Aggregation Results

The final high-N secure aggregation path was tested with 1000 simulated meters.

The value formula was deterministic and varied per meter:

```
value_i = 100 + ((37*i + 11*i*i + 73) mod 900)
```

For `N = 1000`, the expected aggregate was:

```
536600
```

Placement was proportional to observed hardware capacity:

```
Pi 5 = 900 meters
Pi 2 = 100 meters
```

Resource caps:

```
Per-meter memory cap = 16384 KB
Per-meter FD cap = 64
Per-meter CPU time cap = 10 sec
Pi 5 group cap = CPUQuota=300%, MemoryMax=1024M
Pi 2 group cap = CPUQuota=100%, MemoryMax=256M
```

Results collected:

Run ID	N	Pi 5 meters	Pi 2 meters	Expected	Observed	Received	Wall ms	Aggregator RSS KB	Sta
20260606_094119	1000	900	100	536600	missing	missing	missing	missing	FAI
20260606_094449	1000	900	100	536600	536600	1000	10585.106	1056	PA
20260606_094643	1000	900	100	536600	536600	1000	10762.754	1072	PA
20260606_094746	1000	900	100	536600	missing	missing	missing	missing	FAI

The first failed run should be treated as a setup failure. It occurred before the root path issue in the systemd launcher was fixed. The launcher tried to access:

```
/root/mpc-smartgrid
```

instead of the actual user repository path.

The two middle runs are valid successful stress-test results. Both received all 1000 submissions and reconstructed the expected aggregate.

The final run failed after two successes. The logs show that meter logs existed for 900 Pi 5 meters and 100 Pi 2 meters, but the result file lacked observed aggregate and received count. The recorded reason included Pi 2 launch timeout and aggregate mismatch due to missing server result fields. This indicates the high-concurrency path is not yet fully robust under repeated execution. It may be an orchestration timeout, cleanup issue, cgroup scheduling issue, or aggregator completion timing issue. It should not be ignored.

The strongest supported high-N result is:

```
The testbed successfully completed two 1000-meter secure aggregation runs with  
all submissions received and the correct aggregate reconstructed.
```

The more careful reliability statement is:

The 1000-meter secure aggregation path is feasible on the current two-Pi setup, but repeated-run reliability still needs improvement because one later run failed due to launch or timeout behavior.

8.6 Throughput of Successful 1000-Meter Runs

Successful run 1:

1000 submissions in 10585.106 ms
throughput $\approx$ 94.47 submissions/sec

Successful run 2:

1000 submissions in 10762.754 ms
throughput $\approx$ 92.91 submissions/sec

Average over successful runs:

average wall time $\approx$ 10673.93 ms
average throughput $\approx$ 93.69 submissions/sec

Aggregator memory footprint was extremely small:

1056 KB and 1072 KB RSS

This suggests the central collector itself is not the bottleneck. Bottlenecks are more likely client launch rate, process scheduling, TCP connection bursts, Pi 2 cgroup limits, or orchestration timeouts.

9. Meaning of Results

9.1 Correctness

The all-party MPC tests demonstrate that the C MPC implementation computes correct aggregates for the tested 2-party and 6-party deployments. The smart-meter emulator demonstrates that real input files can be used as the private value source without changing protocol behavior.

The secure aggregation stress path demonstrates that a central collector can receive masked submissions from 1000 simulated meters and recover the correct aggregate in successful runs.

9.2 Resource Viability

The 2-party smart-meter workload is lightweight in CPU and memory terms. Even under CPUQuota=25% and MemoryMax=16M, correctness was preserved. This means the current 2-party workload is viable on Pi-class hardware.

The 1000-meter secure aggregation run is more stressful. It launches many constrained processes, but the aggregator remains small. The successful 1000-meter runs show that the system can emulate a large smart-meter fleet at process level if load is placed proportionally across Pi 5 and Pi 2.

9.3 Pi 5 versus Pi 2 Placement

A 50/50 split would overload Pi 2 and produce misleading results. The final test used proportional placement:

```
Pi 5 = 900 meters
Pi 2 = 100 meters
```

This is appropriate because Pi 5 had substantially more available memory:

```
Pi 5 MemAvailable ≈ 7563-7572 MB
Pi 2 MemAvailable ≈ 620-623 MB
```

The final placement avoided making the experiment primarily a Pi 2 process-capacity test. It still kept Pi 2 active as a constrained secondary host.

9.4 Protocol Distinction

The work now includes two different privacy-preserving aggregation models.

The first is all-party MPC:

```
Every party exchanges shares with every other party.
Every party reconstructs the final aggregate.
Communication grows roughly  $O(N^2)$ .
Validated for  $N=2$  and  $N=6$ .
```

The second is central secure aggregation:

```
Each meter sends one masked value to a central collector.
Pairwise masks cancel in the final sum.
The central collector reconstructs only the aggregate.
```

Communication to the collector grows roughly $O(N)$.
Validated successfully for $N=1000$ in two runs.

These are not the same protocol. The 1000-meter result should not be described as 1000-party all-party MPC. It is a 1000-client central secure aggregation stress test.

10. Failures and Debugging Lessons

Several important failures occurred during the project. They are worth documenting because they clarify system maturity.

10.1 Tailscale Tailnet and SSH Failures

Early failures were caused by stale Tailscale state, expired keys, and tailnet confusion. Symptoms included ping timeout, SSH timeout, asymmetric connectivity, and devices visible in `tailscale status` but unreachable.

The system was recovered by forcing Pi 2 into a clean Tailscale state and using a new working address.

10.2 Binary Architecture Contamination

A sync operation copied PC 5 build artifacts to the Pis. This caused execution errors because PC 5 binaries are x86_64 while Pi binaries must be ARM.

The fix was to exclude `build/` from all sync commands:

```
rsync --exclude build/
```

and rebuild locally on each Pi.

10.3 `systemd-run` Non-Interactive Issues

Automated constrained tests initially hung because `systemd-run` required authentication in non-interactive SSH contexts. Passwordless use of `systemd-run` was configured through sudoers for lab testing.

10.4 `systemd-run --scope --wait` Conflict

The benchmark worker originally used incompatible flags:

```
systemd-run --scope --wait
```

This failed because `--wait` may not be combined with `--scope`. Removing `--wait` fixed the worker.

10.5 Root Home Directory Issue in Cgroup Launcher

When launching meter groups through `systemd-run`, the launch script executed as root. It used `$HOME/mpc-smartgrid`, which resolved to:

```
/root/mpc-smartgrid
```

The actual repo was under the user home directory. The fix was to resolve the script path and derive the repo directory from that path.

10.6 1000-Meter Repeat Failure

After two successful 1000-meter secure aggregation runs, one repeat failed. Since meter logs were present but aggregate result fields were missing, this likely reflects orchestration timeout or server completion timing rather than a proven cryptographic failure. It must be investigated with more detailed logging before making strong reliability claims.

11. Validated Claims

The following claims are supported by experimental evidence.

1. Pi 5 and Pi 2 can communicate over Tailscale and execute distributed TCP-based MPC experiments.
2. The C MPC implementation correctly reconstructs aggregates in the tested 2-party and 6-party cases.
3. The smart-meter emulator can read CSV meter values and feed them into the same MPC protocol path.
4. The refactored `mpc_runner` enables both benchmark and smart-meter execution through shared protocol logic.
5. The 2-party smart-meter workload remains correct under CPU and memory constraints on Pi 5.
6. The automated benchmark suite can collect structured latency, RSS, and correctness results.
7. A central secure aggregation stress path can process 1000 simulated constrained meters successfully in at least two full runs.
8. The central aggregator can reconstruct the expected aggregate from 1000 masked submissions while using approximately 1 MB RSS.

12. Claims Not Yet Supported

The following claims are not yet proven.

1. The system is not yet proven as a 1000-party all-party MPC protocol. The 1000-meter experiment is central secure aggregation, not all-party MPC.
2. The system is not yet proven reliable over many 1000-meter repetitions. Two runs passed, one later run failed.
3. The system is not yet dropout-resilient. If clients fail to submit, the current secure aggregation protocol does not recover the missing masks.

4. The system is not yet secure against sophisticated collusion models.
5. The system is not yet an embedded STM32/MSP430 deployment.
6. The Tailscale network does not represent real smart-grid radio or field-network conditions.
7. The resource caps are Linux process and cgroup approximations, not microcontroller-level constraints.
8. Network impairment testing using `tc netem` has not yet been completed.
9. Long-duration operation over hours or days has not yet been validated.

13. Recommended Next Tests

13.1 Repeat 1000-Meter Secure Aggregation

Run at least 10 trials at N=1000 with the same placement and constraints:

```
N = 1000
Pi 5 = 900 meters
Pi 2 = 100 meters
per-meter mem = 16 MB
Pi 5 group = CPU 300%, Memory 1024M
Pi 2 group = CPU 100%, Memory 256M
```

Record:

```
pass/fail
received count
expected aggregate
observed aggregate
wall time
aggregator RSS
failure reason
```

This converts the current result from feasibility evidence into reliability evidence.

13.2 Investigate Third-Run Failure

For the failed 1000-meter run, add:

```
server timeout reason
server exit code
number of accepted connections over time
number of meter process failures
per-Pi launch completion timestamp
```

```
Pi 2 cgroup memory and CPU pressure  
kernel OOM logs
```

Useful commands:

```
journalctl  
dmesg  
systemctl status transient units  
ps and free snapshots during run
```

13.3 N-Scaling Secure Aggregation

Run:

```
N = 250, 500, 750, 1000, 1250, 1500
```

Use proportional placement. Stop at first consistent failure. This finds the real capacity curve.

13.4 Memory Sweep

For N=1000, test:

```
per-meter memory = 32 MB  
per-meter memory = 16 MB  
per-meter memory = 8 MB  
per-meter memory = 4 MB
```

This identifies the minimum viable per-meter memory cap.

13.5 Pi 2 Load Sweep

Keep N=1000 and vary Pi 2 participation:

```
Pi 2 = 50 meters  
Pi 2 = 100 meters  
Pi 2 = 150 meters  
Pi 2 = 200 meters  
Pi 2 = 300 meters
```

This determines how much load Pi 2 can reasonably carry.

13.6 Network Impairment

Use `tc netem` to test:

```
100 ms delay
100 ms delay + 1% loss
100 ms delay + 3% loss
```

Run both 2-party MPC and 1000-meter secure aggregation under impaired network conditions.

13.7 Dropout Simulation

In real smart-meter networks, some meters fail to submit. Test:

```
1% dropout
5% dropout
10% dropout
```

The current secure aggregation design will not handle dropouts correctly if masks are not recovered. This should be documented and then addressed with a dropout-resilient protocol.

13.8 Embedded Transport Preparation

The code should continue isolating the communication backend behind a stable interface. Future embedded work should replace TCP with:

```
UART
LoRa
BLE
802.15.4
gateway-mediated serial transport
```

without changing field arithmetic, sharing logic, or high-level aggregation semantics.

14. Final Technical Assessment

The project has progressed from a fragile network setup to a functional distributed privacy-preserving aggregation testbed. The small-N all-party MPC path is correct for the tested cases. The smart-meter emulator path is correct and works under CPU/memory constraints. The automated benchmarking infrastructure is operational. The high-N secure aggregation path achieved two successful 1000-meter runs with proportional load placement and constrained meter processes.

The current strongest result is:

1000 simulated constrained meters, placed 900 on Pi 5 and 100 on Pi 2, submitted masked readings to a Pi 5 aggregator. The aggregator received all 1000 submissions and reconstructed the expected aggregate of 536600 in approximately 10.6 to 10.8 seconds, using about 1 MB RSS.

The current main weakness is repeated-run reliability at high concurrency. A later run failed due to launch or timeout behavior. This does not invalidate the successful runs, but it prevents a strong reliability claim until repeated trials are stable.

The system should now be described as:

A working Raspberry Pi based distributed validation platform for C-based MPC and secure aggregation experiments, with demonstrated correctness under small all-party MPC, constrained smart-meter emulation, and successful high-N secure aggregation stress trials.

It should not yet be described as:

- A production-ready smart-grid MPC system.
- A validated 1000-party all-party MPC implementation.
- A completed embedded microcontroller deployment.
- A dropout-resilient secure aggregation protocol.

15. Conclusion

The Raspberry Pi testbed successfully validates the core engineering path for privacy-preserving smart-meter aggregation. Pi 5 and Pi 2 communicate over Tailscale, build and run the C MPC library locally, execute distributed aggregation protocols, and produce correct aggregate outputs. The system supports both benchmark-mode MPC and CSV-backed smart-meter emulation. Automated resource-constrained tests show that the 2-party smart-meter workload remains correct under CPU and memory limits.

The final secure aggregation stress test substantially extends the evaluation. With proportional placement, 900 simulated meters on Pi 5 and 100 on Pi 2, the system successfully processed 1000 constrained masked submissions and reconstructed the correct aggregate in two full runs. This demonstrates high-N feasibility for a central secure aggregation model on limited hardware. A later failed run highlights the need for further reliability work, especially around process launching, cgroup scheduling, timeouts, and repeated-run stability.

The next phase should focus on repeatability, N-scaling, network impairment, dropout handling, and transport abstraction. Those tests will determine whether the system remains viable beyond a controlled lab run and whether it can be credibly moved toward embedded smart-meter hardware.